

# Clarion Coach

AI-Powered Real-Time Customer Support Coaching Assistant - a multi-agent, RAG-grounded platform that coaches support agents turn-by-turn instead of reviewing them after the fact.

FastAPI

React + TypeScript + Vite

ChromaDB / RAG

Google Gemini

Multi-Agent Orchestration

**Project Documentation** Technical Architecture & Design Report

**Prepared by** Sangaraju Suvida

**Project Structure** Four-Milestone Development

**Architecture** Code-Based Multi-Agent System + RAG

# Table of Contents

A structured technical report for mentor review, repository documentation, and project walkthrough.

| SECTION | TOPIC                                   | PAGE |
|---------|-----------------------------------------|------|
| 01      | Executive Summary                       | 3    |
| 02      | Problem Statement & Objectives          | 4    |
| 03      | System Architecture                     | 5    |
| 04      | Interaction Modes                       | 6    |
| 05      | Multi-Agent Pipeline                    | 7    |
| 06      | Technology Stack                        | 9    |
| 07      | Data Model & Application Flow           | 10   |
| 08      | RAG Knowledge Pipeline                  | 11   |
| 09      | API & Backend Contract                  | 12   |
| 10      | Frontend & Live Coaching Console        | 13   |
| 11      | Development Timeline & Four Milestones  | 14   |
| 12      | Engineering Challenges & Fixes          | 15   |
| 13      | Evaluation Criteria Mapping             | 16   |
| 14      | Security, Testing & Future Enhancements | 17   |

## About this document

This report documents Clarion Coach as implemented for the project review. It is intentionally more detailed than a README and explains the architecture, agent responsibilities, data flow, RAG pipeline, API contract, milestones, engineering fixes, evaluation approach, and testing strategy.

# Executive Summary

Clarion Coach is an AI-powered customer support coaching platform designed to help support representatives improve the quality of their responses while a conversation is taking place. The system combines conversation analysis, semantic knowledge retrieval, LLM-based coaching, escalation-risk monitoring, and multi-turn simulation.

The architecture is intentionally code-based: a Python Orchestrator coordinates specialized agents through function calls and structured Pydantic objects. The design keeps individual responsibilities narrow and makes the complete workflow easier to inspect, test, and extend.

| CAPABILITY           | PROJECT IMPLEMENTATION                                                      |
|----------------------|-----------------------------------------------------------------------------|
| Multi-agent workflow | Specialized agents coordinated by a Python Orchestrator                     |
| Real-time coaching   | Suggested response, communication notes and score per turn                  |
| Grounded knowledge   | ChromaDB semantic retrieval with Sentence Transformer embeddings            |
| Training modes       | Simulator, Manual and Replay                                                |
| Risk awareness       | Escalation probability, level, reasoning and recommended action             |
| Response safety      | Rules against invented names, identifiers and unsupported completed actions |

## Core value proposition

Traditional coaching can be retrospective. Clarion Coach moves useful feedback closer to the point of action: the current conversation turn. The system therefore treats the conversation as a sequence of stateful turns rather than a collection of independent messages.

The project was developed across four milestones, progressing from foundation and orchestration to AI/RAG integration, coaching functionality, and final reliability hardening.

# Problem Statement & Objectives

## The Problem

Customer-support representatives often need to understand the customer issue, recognize emotional cues, search a knowledge base, and compose a professional reply simultaneously. This creates cognitive load and can lead to slower responses, inconsistent communication, or unsupported assumptions.

## Project Objective

Develop an AI-powered support coaching assistant that analyses customer conversations, retrieves relevant knowledge, generates a customer-ready response recommendation, evaluates communication quality, and monitors escalation risk.

## Objectives

- Detect customer intent and emotional signals.
- Track sentiment, frustration and urgency as conversation context.
- Retrieve domain knowledge using semantic search and RAG.
- Generate concise and context-aware response suggestions.
- Evaluate tone, clarity, grammar, professionalism and empathy.
- Monitor escalation risk and provide reasoning.
- Support realistic practice using Simulator, Manual and Replay modes.
- Reduce hallucinated names, identifiers, unsupported policies and unsupported completed actions.

## Implemented Modules

| MODULE                      | PURPOSE                                                                     |
|-----------------------------|-----------------------------------------------------------------------------|
| Session Management          | Creates and maintains the active coaching session and conversation context. |
| Knowledge Management        | Uploads and indexes support documents for semantic retrieval.               |
| Agent Orchestration         | Coordinates the specialized AI agents in a defined workflow.                |
| Live Coaching Console       | Displays conversation, coaching, knowledge and risk information.            |
| Simulator / Manual / Replay | Provides three modes for practice, direct coaching and transcript review.   |
| Reporting                   | Supports session-level reporting and PDF export.                            |

# System Architecture

Clarion Coach follows a layered architecture with a React/Vite frontend, a FastAPI backend, a Python orchestration layer, specialized agents, Gemini for LLM generation, and ChromaDB for semantic knowledge retrieval.

| FRONTEND                           | FastAPI API                                 | ORCHESTRATOR                             |
|------------------------------------|---------------------------------------------|------------------------------------------|
| React / TypeScript / Vite          | /chat + session + knowledge + report routes | Mode routing + context + agent execution |
| Conversation UI                    | Pydantic validation                         | Structured Pydantic outputs              |
| Coaching / Knowledge / Risk panels | JSON response                               | Execution trace                          |

## Agent Communication

Agents do not directly call one another. The Orchestrator calls a Python function, stores the returned object in a local variable, and passes that object or its relevant fields to the next component.

For example:

```
analysis = analyze_intent(customer_message)
knowledge = recommend(customer_message)
coaching = coach(customer_message, analysis, ...)
risk = monitor(customer_message, analysis, ...)
```

## Response Contract

The major structured models include IntentAnalysis, RetrievedChunk, CoachingSuggestion, EscalationRisk, and ChatTurnResponse. FastAPI serializes the final Pydantic response into JSON for the frontend.

## Resilience

The orchestration design isolates agent failures and uses structured results so that a single model or retrieval problem can be handled without unnecessarily breaking the complete conversation turn.

# Interaction Modes

All three interaction modes feed the same core coaching concepts. The main difference is where the current customer message comes from.

| MODE      | INPUT SOURCE                     | PIPELINE BEHAVIOUR                                                                         | PRIMARY USE                       |
|-----------|----------------------------------|--------------------------------------------------------------------------------------------|-----------------------------------|
| Simulator | Customer Simulator Agent         | Simulator creates the customer turn, then Intent -> Knowledge -> Coaching -> Escalation    | Self-contained training           |
| Manual    | Trainee-entered customer message | Customer message goes directly to Intent -> Knowledge -> Coaching -> Escalation            | Direct message coaching           |
| Replay    | Stored transcript                | Customer turns use the same pipeline; non-customer lines can be recorded as replay context | Historical review / demonstration |

## Simulator Mode

Simulator Mode creates a realistic practice loop. The selected persona, scenario, product, difficulty, previous customer turns, and the latest support reply influence the next simulated customer message. Emotional state is progressed gradually so the simulator does not reset to a new state on every turn.

## Manual Mode

Manual Mode skips customer simulation. The trainee supplies the customer message directly, allowing the coaching system to be used against a specific issue without requiring a generated customer turn.

## Replay Mode

Replay Mode processes an existing conversation transcript step by step. This supports retrospective coaching and allows a reviewer to demonstrate how the same analysis pipeline can be applied to previously recorded interactions.

# Multi-Agent Pipeline

The system is composed of specialized agents with narrow responsibilities. The Orchestrator controls their execution and aggregates their outputs.

| STAGE        | AGENT                    | RESPONSIBILITY                                            | PRIMARY OUTPUT                               |
|--------------|--------------------------|-----------------------------------------------------------|----------------------------------------------|
| 1            | Customer Simulator       | Generate realistic customer turn during Simulator Mode.   | customer_message, emotion, frustration_level |
| 2            | Intent Analysis          | Detect intent and emotional state.                        | IntentAnalysis                               |
| 3a           | Knowledge Recommendation | Retrieve relevant knowledge from ChromaDB.                | RetrievedChunk list                          |
| 3b           | Escalation Monitor       | Estimate escalation risk from conversation context.       | EscalationRisk                               |
| 4            | Coaching                 | Generate customer-ready draft and communication feedback. | CoachingSuggestion                           |
| Post-session | Reporting / Summary      | Build session-level report and recommendations.           | Report data                                  |

## Coaching Inputs

- Current customer message and latest agent response.
- Intent, sentiment, frustration and urgency.
- Conversation history and session context.
- Previous suggested responses and coaching tips.
- Retrieved knowledge titles and previews.
- Logged-in support representative name when supplied by the application.

## Dynamic Behaviour

The Coaching Agent is designed to adapt to the current turn rather than return a fixed script. High frustration should lead to stronger acknowledgement and a concrete next step; resolving conversations should shift toward an appropriate closing tone.

## Anti-Hallucination Behaviour

Customer-facing drafts are constrained against invented names, order IDs, transaction IDs, tracking numbers, unsupported product facts, unsupported policy names, unsupported dates, and claims that an action has already been completed when the application has not confirmed it.

# Technology Stack

| LAYER          | TECHNOLOGY                               | PURPOSE                                                               |
|----------------|------------------------------------------|-----------------------------------------------------------------------|
| Frontend       | React + TypeScript + Vite                | Single-page application, UI components and development/build workflow |
| Styling        | Tailwind CSS                             | Consistent styling across the coaching interface                      |
| Backend        | Python + FastAPI + Uvicorn               | REST API, request handling and application server                     |
| Validation     | Pydantic                                 | Structured request/response models and validation                     |
| LLM            | Google Gemini                            | LLM generation for analysis/coaching tasks                            |
| Embeddings     | Sentence Transformers - all-MiniLM-L6-v2 | Vector representations for semantic retrieval                         |
| Vector Store   | ChromaDB                                 | Persistent semantic knowledge retrieval                               |
| Reporting      | ReportLab                                | PDF report generation                                                 |
| Source Control | Git + GitHub                             | Version control and project submission                                |

## Why this stack

React/Vite provides a fast development experience for the coaching console. FastAPI gives a compact Python API layer that works naturally with Pydantic models. Sentence Transformers and ChromaDB provide a lightweight semantic retrieval pipeline. Gemini supplies the LLM layer used by the project's AI agents.



# Data Model & Application Flow

The application uses structured request and response models to keep the data passed between the frontend, Orchestrator and agents predictable.

| MODEL              | ROLE                    | IMPORTANT FIELDS                                                   |
|--------------------|-------------------------|--------------------------------------------------------------------|
| ChatRequest        | Incoming chat request   | session_id, message, role                                          |
| ChatMessage        | Conversation message    | id, role, content, timestamp                                       |
| IntentAnalysis     | Customer analysis       | intent, sentiment, frustration, urgency, confidence                |
| CoachingSuggestion | Coaching result         | suggested_response, notes, scores, coaching_score, score_reasoning |
| RetrievedChunk     | Knowledge result        | id, source, title, preview, similarity, type                       |
| EscalationRisk     | Risk result             | probability, level, reasoning, recommended_action                  |
| ChatTurnResponse   | Aggregated API response | turn, analysis, coaching, knowledge, risk, agent_trace             |

## Request-to-Response Flow

Frontend JSON request -> FastAPI validates it as ChatRequest -> Orchestrator builds session context -> agents execute -> results are stored in variables -> Orchestrator constructs ChatTurnResponse -> FastAPI serializes it to JSON -> frontend maps the returned fields to the live console.

## Session Context

- Conversation history from previous turns.
- Previous suggested responses and coaching tips.
- Frustration and intent progression.
- Current turn number.
- Last agent message.
- Logged-in support representative identity where provided.

# RAG Knowledge Pipeline

The RAG layer gives the coaching workflow access to project/domain knowledge instead of requiring the LLM to answer every support question from general model knowledge.

| INGESTION PHASE | PROCESS                                   |
|-----------------|-------------------------------------------|
| 1               | Upload a supported knowledge document.    |
| 2               | Extract document text.                    |
| 3               | Split text into manageable chunks.        |
| 4               | Generate Sentence Transformer embeddings. |
| 5               | Store chunks and vectors in ChromaDB.     |

| RETRIEVAL PHASE | PROCESS                                            |
|-----------------|----------------------------------------------------|
| 1               | Receive the current customer query.                |
| 2               | Represent the query in the same embedding space.   |
| 3               | Perform semantic similarity search in ChromaDB.    |
| 4               | Return relevant chunks and similarity information. |
| 5               | Pass relevant context to the Coaching Agent.       |

## Semantic Search

Semantic retrieval matches meaning rather than requiring exact keyword overlap. This is useful in support environments where a customer can describe an issue differently from the wording used in a knowledge document.

## Grounding Rule

Retrieved knowledge is supporting context, not permission to invent facts. When relevant knowledge is not available, the response should not manufacture a policy, timeline, identifier, or system action.

# API & Backend Contract

| METHOD | ENDPOINT          | PURPOSE                                                           |
|--------|-------------------|-------------------------------------------------------------------|
| POST   | /chat             | Run a customer-support turn through the current backend pipeline. |
| GET    | /docs             | Open FastAPI Swagger/OpenAPI documentation.                       |
| POST   | /upload           | Upload a knowledge-base document using the project API.           |
| POST   | /knowledge/search | Perform semantic knowledge search.                                |
| POST   | /session/start    | Start a new coaching session.                                     |
| GET    | /session/latest   | Retrieve the latest session.                                      |

## Example Chat Request

```
{
  "session_id": "session-id",
  "message": "My payment failed.",
  "role": "customer"
}
```

## Typical Response Structure

The returned JSON can contain the current turn, intent analysis, coaching result, knowledge recommendations, risk information, conversation summary and agent trace. The frontend uses this single response to update the coaching console.

## Frontend API Reliability

The final integration includes explicit handling for unreachable backend errors, non-success status codes, invalid JSON responses and request timeout. Knowledge upload handling also reports failed HTTP responses instead of treating them as successful results.

# Frontend & Live Coaching Console

The frontend presents the system as a practical support-training console. The central experience is the conversation view plus coaching, knowledge, and risk information.

| AREA          | RESPONSIBILITY                                                             |
|---------------|----------------------------------------------------------------------------|
| Session setup | Select or configure the interaction mode and session context.              |
| Conversation  | Display customer and support-agent messages in sequence.                   |
| Coaching      | Show suggested response, communication notes, scores and improvement tips. |
| Knowledge     | Show relevant retrieved support information.                               |
| Risk          | Show escalation probability/level, reasoning and recommended action.       |
| Trace         | Show agent execution details when available.                               |
| Reports       | Access post-session reporting and PDF output.                              |

## Logged-in Agent Identity

The signed-in support representative name can flow from the frontend user layer through the API request and Orchestrator into the Coaching Agent as `agent_name`. This value is distinct from customer identity and must never be substituted for the customer's name.

## Real vs Demo API Mode

The API layer can distinguish between the real backend path and the demo/mock path through the configured API base URL. When the real URL is configured, chat-related results are obtained from the FastAPI backend instead of local mock responses.

# Development Timeline & Four Milestones

| MILESTONE   | FOCUS                  | MAJOR WORK                                                                                                                                                 |
|-------------|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Milestone 1 | Foundation             | React/Vite frontend, FastAPI backend, Pydantic schemas, session handling, REST API, code-based Orchestrator.                                               |
| Milestone 2 | AI + Knowledge         | Intent analysis, sentiment/frustration analysis, document processing, embeddings, ChromaDB, semantic retrieval, Gemini integration.                        |
| Milestone 3 | Coaching Experience    | Coaching Agent, Escalation Monitor, Customer Simulator, Manual Mode, Replay Mode, session context and execution trace.                                     |
| Milestone 4 | Quality + Finalization | Dynamic responses, anti-hallucination controls, agent-name flow, unsupported-action prevention, API error handling, reporting, security and documentation. |

## Milestone Traceability

Each milestone advances the system from a technical foundation toward a complete training product. The final milestone focuses on correctness and reliability rather than changing the architecture.

## Agile Documentation

The accompanying Agile Template groups user stories under the same four milestones and tracks dependencies, MoSCoW priority, assignee and completion status. This keeps the project documentation aligned with the implementation history.

# Engineering Challenges & Fixes

| CHALLENGE                                   | FIX / ENGINEERING RESPONSE                                                                                |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Hallucinated customer names                 | Strengthened customer-identity rules and validation so names are used only when explicitly available.     |
| Hallucinated order/transaction IDs          | Added identifier grounding and validation; real customer-supplied identifiers are preserved.              |
| Unsupported completed actions               | Changed response behaviour to recommend supported next steps unless an action is actually confirmed.      |
| Repeated responses                          | Provided previous suggestions/tips and dynamic context to reduce unnecessary repetition.                  |
| Backend API errors hidden by demo behaviour | Added explicit timeout, unreachable-backend, non-2xx and invalid-JSON handling in the frontend API layer. |
| Knowledge upload error handling             | Changed failed uploads to raise an error instead of treating an error payload as successful JSON.         |
| Environment compatibility                   | Used a compatible local Python virtual environment for the pinned AI/RAG dependencies.                    |

## Why these fixes matter

The most important engineering lesson from the project was that prompt quality alone is not enough. Conversation grounding, structured data contracts, validation, error handling and end-to-end testing all contribute to reliable AI behaviour.

# Evaluation Criteria Mapping

| EVALUATION AREA            | HOW CLARION COACH ADDRESSES IT                                                                   |
|----------------------------|--------------------------------------------------------------------------------------------------|
| Realistic simulation       | Persona, scenario, product, history and evolving frustration are provided to the simulator.      |
| Intent and sentiment       | Structured IntentAnalysis captures intent, sentiment, frustration, urgency and confidence.       |
| Knowledge relevance        | Semantic vector retrieval returns relevant chunks with source and similarity information.        |
| Coaching usefulness        | Suggested response, communication notes, scores and improvement tips are returned per turn.      |
| Escalation awareness       | Risk probability, level, reasoning and recommended action are provided.                          |
| Multi-turn consistency     | History, previous suggestions, previous tips and emotional context are retained.                 |
| Safety / grounding         | Names, identifiers, unsupported policies and unsupported completed actions are constrained.      |
| Mode coverage              | Simulator, Manual and Replay provide different entry points into the shared workflow.            |
| Documentation completeness | Architecture, agents, modes, RAG, API, milestones, testing and engineering fixes are documented. |

## Project Quality Perspective

The evaluation strategy focuses not only on whether the model can generate text, but whether the generated output is useful, grounded, context-aware, appropriately scored, and integrated into a working application.

# Security, Testing & Future Enhancements

## Security

- Gemini API credentials remain in backend environment configuration.
- Real secrets must not be committed to GitHub.
- The frontend should never receive the Gemini API key.
- Generated customer-facing text should not expose sensitive authentication or payment information.
- Local vector-store/cache data should remain excluded from source control when configured as generated data.

## Testing Strategy

| TEST AREA               | WHAT IS VERIFIED                                                                          |
|-------------------------|-------------------------------------------------------------------------------------------|
| Intent                  | Structured intent/sentiment output is returned for a valid customer message.              |
| RAG                     | Relevant knowledge can be retrieved and used as coaching context.                         |
| Anti-hallucination      | No invented customer name or identifier when none is available.                           |
| Identifier preservation | An explicitly supplied identifier is not replaced by another value.                       |
| Unsupported actions     | The system does not claim a refund/reset/dispatch was completed without confirmation.     |
| Dynamic response        | Five-turn interaction produces context-specific responses without unnecessary repetition. |
| Agent identity          | Logged-in agent name reaches coaching separately from customer identity.                  |
| API reliability         | Backend timeout and failed HTTP/JSON responses are surfaced clearly.                      |
| Security                | No API key is hardcoded or exposed in the frontend.                                       |

## Future Enhancements

- More advanced cross-session analytics and performance trends.
- Role-based authentication for multi-user support teams.
- Per-document retrieval filtering and richer knowledge metadata.
- Production chat integration for live support environments.
- Personalized coaching profiles based on recurring communication patterns.
- Expanded server-side validation and response-quality monitoring.

## Conclusion

Clarion Coach demonstrates how a small set of specialized AI components can be coordinated into a practical customer-support coaching application. The combination of a code-based Orchestrator, semantic retrieval, LLM-based coaching, session context and escalation monitoring provides a structured path from a raw customer message to an actionable support recommendation.

The four-milestone development structure kept the implementation focused: foundation, AI/RAG pipeline, coaching experience, and final quality hardening. The resulting documentation is intended to make the system understandable not only as a collection of features, but as an engineered workflow with defined responsibilities, interfaces, safeguards and validation criteria.



Project status: Functionally complete and prepared for final project review, with live-environment verification performed separately using the configured local backend and Gemini credentials.